

## 효과적인 답변을 얻기 위한 고급 프롬프팅 기법 (링크 : [프롬프트 작성 챗봇](#))

역할부여+멀티턴 몰아가기 :  챗GPT가 말을 잘 듣게 만드는 프롬프트 작성방법

GPTs 활용 :  프롬프트 ‘이것’ 하나가 결정합니다. | 나만의 프롬프트 비서 만들기

챗GPT 프로젝트 폴더 활용

 챗GPT·Claude·Gemini 공식문서까지 썩 다 파봤습니다. | 프롬프트 작성 Bible 제공

### 1. 일반 원칙: 프롬프트 작성의 기본 원칙 및 유의사항

**프롬프트 엔지니어링(prompt engineering)**이란 모델이 원하는 결과를 안정적으로 생성하도록 입력 프롬프트를 설계하고 최적화하는 과정이다. 모델 출력은 확실적인 생성에 기반하므로, 원하는 결과를 얻기 위해서는 명확한 지침과 충분한 맥락을 제공하는 것이 핵심입니다. 아래에 프롬프트 작성 시 보편적으로 유념해야 할 기본 원칙들을 정리했습니다.

- **명확하고 구체적으로 작성**: 프롬프트는 모호하지 않고 구체적이어야 합니다. 모델이 이해하기에 충분한 맥락 (**context**)과 세부 정보를 포함시키세요. 모호한 질문 대신 정확히 무엇을 원하는지 지정해야 합니다. 예를 들어 "설명해줘"보다는 "양자 컴퓨팅의 원리를 비전공자도 이해하도록 3문단으로 설명해줘"처럼 작성합니다. 또한 필요한 경우 원하는 출력 형식이나 분량도 지정하세요 (예: "한 줄 요약" 또는 "표 형태로 답변").
- **역할과 구체적 지시 제공**: 모델에게 특정 역할(**persona**)이나 목표를 부여하면 보다 일관된 톤과 관점을 얻을 수 있습니다. 예를 들어 "당신은 숙련된 번역가입니다..."와 같이 시작하여 원하는 스타일이나 전문성 수준을 명시하세요. 또한 동작을 나타내는 동사로 직접적인 지시를 내리는 것이 효과적입니다 (예: "요약하라", "비교하라", "분류하라" 등). 톤이나 스타일도 필요한 경우 "격식체로 답변해줘", "친근하고 유머러스하게 설명해줘"처럼 지정할 수 있습니다. 한 프롬프트 안에서는 상충되는 지시를 피하고, 일관된 방향으로 안내해야 합니다.
- **맥락과 제약조건 제시**: 모델이 따라야 할 제약 사항이나 참고해야 할 정보 자료가 있다면 프롬프트에 명시합니다. 예를 들어 문서 요약을 원한다면 해당 문서 내용을 프롬프트에 포함하거나, 외부 지식이 필요한 경우 관련 정보를 제공해야 합니다. 모델이 사용해서는 안 되는 정보나 형식도 있다면 (예: 내부 정보 노출 금지 등) 함께 언급합니다. 프롬프트에 구체적으로 "위 내용만 참고해서 답변해" 또는 "모르는 내용은 짐작으로 쓰지 마"와 같이 적어 모델이 근거 없는 추측(환각)을 하지 않도록 유도할 수 있습니다.
- **출력 형식 지정**: 모델이 답변을 어떤 형식(**format**)으로 주길 원하는지 알려주는 것이 좋습니다. 글머리 기호 목록, 표, JSON, 마크다운 등의 형식을 명시하면 모델이 그 구조를 따릅니다. 예를 들어 "중요 항목을 번호 목록으로 제공해줘",

"JSON 형식으로 출력해줘."처럼 요구할 수 있습니다. 프롬프트에 예상 출력 예시를 간단히 포함 하는 것도 모델의 이해를 높이는 데 도움이 됩니다.

- **간결성과 핵심 전달**: 필요 이상의 중복이나 장황한 설명을 피하고, 핵심만 간결히 전달합니다. 특히 시스템/지시 메시지를 사용할 수 있는 환경이라면 거기에 상세 지침을 적고, 사용자 프롬프트는 필요한 입력이나 질문만 담는 편이 효율적입니다. 단, 너무 짧아서 애매한 프롬프트는 피해야 합니다. 예를 들어 "자료 분석해줘."보다는 **1•2•3 4 5 6•••1** "다음 매출 데이터셋에서 월별 추이를 분석하고, 상승/하락 요인을 설명해줘."처럼 맥락과 기대 결과를 모두 포함합니다.
- **예시 활용 (Few-shot 프롬프팅)**: 모델이 따라야 할 출력의 형식이나 스타일이 있다면 예시를 함께 제공하는 것이 유용합니다. 두세 개의 **Q&A** 쌍이나 입력-출력 예시를 프롬프트에 포함하면 모델이 패턴을 학습하여 일관된 형식의 결과를 생성합니다. 이는 특히 목록 작성, 번역 어투 결정, 코드 출력 형식 등에서 효과적입니다. 단, 예시는 너무 많이 주지 말고 **1~3개** 정도가 적당하며, 예시와 실제 요청은 구분이 되도록 명확히 표기합니다 (필요하면 구분선이나 <예시> 태그 등을 사용).
- **모델 한계 인지 및 반복 개선**: 처음부터 완벽한 프롬프트를 얻기 어렵습니다. 반복(**iterative**) 접근을 통해 결과를 확인하면서 프롬프트를 다듬어 나갑니다. 모델의 응답을 검토하여 부족한 부분이 있다면 프롬프트에 조건을 추가하거나 표현을 바꾸어 다시 요청하세요. 예를 들어 첫 응답이 너무 일반적이면 "더 구체적으로 설명해줘.", 중요한 포인트가 빠졌으면 "**X**에 대해 언급해줘." 같이 후속 지시를 합니다. 필요한 경우 한 번에 모든 걸 요구하기보다 단계별로 요청을 나누는 것도 방법입니다. 이런 프롬프트-응답 튜닝 과정을 통해 원하는 출력에 점점 가까워지도록 모델을 안내할 수 있습니다.
- **금지 및 주의사항 명시**: 생성되지 않았으면 하는 금지어 또는 부정 요소가 있다면 프롬프트에 분명히 언급하세요. 예를 들어 이미지 생성 시 "**--no**" 프롬프트나 텍스트 생성 시 "문체는 ~~를 제외하고 사용하지 마" 등으로 부정 프롬프트를 사용할 수 있습니다. 또한 민감하거나 잘못된 정보가 나오지 않도록 "사실과 다른 내용은 생성하지 마.", "출처가 불분명한 내용은 추측으로 쓰지 마." 같이 제약 조건을 걸어두면 모델이 출력에서 해당 요소를 배제하려 노력합니다.

## 2. LLM이 답변하는 방식.

텍스트는 “의미를 고정”하면 언어는 중요하지 않고, 이미지는 “학습된 언어(영어)”를 그대로 쓰는 게 정답

### LLM의 실제 작동 방식(한국어로 프롬프팅이 최선)

- 한국어로 질문해도 영어로 ‘번역 문장’을 만들지는 않음
- 질문은 바로 **\*\*언어 중립적인 의미 벡터(semantic vector)\*\***로 변환됨
- 외국어 문헌 검색도 같은 의미 공간에서 유사도 비교로 수행됨

✓ **한국어 → 의미 벡터 → 유사한 의미의 문헌/지식 → 한국어로 답변 생성**

- 그럼에도 문제가 생기는 이유는 번역이 아니라 “의미 선택입니다. 한국어는 주어 생략, 완곡·평가·감정이 섞인 표현, 맥락 의존 문장이 많기 때문에, 그나마 유사도가 높은 의미 하나”를 모델이 임의로 선택하는 과정에서 사용자가 의도하지 않은 방향으로 생성이 틀어질 수 있음, ➡ 이 현상이 오역처럼 체감되는 것. 즉 LLM의 문제는 번역이 아니라, 한국어 질문이 여러 의미로 해석될 수 있다는 점입니다.

## 이미지·영상 프롬프트(영어 프롬프팅이 최선)

이 부분은 텍스트 답변과 구조가 완전히 다릅니다.

- **이유 1 — 학습 데이터의 절대량 차이** : 이미지·영상 생성 모델은 프롬프트-결과 쌍의 대부분이 영어, 스타일, 구도, 조명, 색감 키워드도 영어 기준으로 학습. 즉, 한국어 프롬프트 → 의미 벡터 변환 시 이미 정보가 한 번 줄어든 상태. 영어 프롬프트 → 직접 학습된 패턴과 매칭
- **이유 2 — 이미지 프롬프트는 “뇌왕스”가 아니라 “토큰 매칭”**. 텍스트 답변은 문맥·의도·논리 추론이 중요, 반면 이미지/영상은 키워드 조합 + 가중치 매칭이 핵심. 예로 “vibrant”, “cinematic lighting”, “shallow depth of field”, “wide angle shot” 이런 표현은 한국어로 풀면 길어지고, 의미 벡터 변환 시 정확도가 오히려 떨어짐
- **이유 3 — 애매함이 허용되지 않음**. 텍스트 답변은 애매해도 설명으로 보완 가능. 이미지/영상은 애매하면 바로 결과물 왜곡. 모델이 “그럴듯한 것 하나”를 찍어버림. ➡ 영어는 이미 의미가 고정된 시각 토큰으로 작동

## 3. LLM 프롬프팅 전략: 텍스트 생성 목적별 기법

**대규모 언어 모델(LLM)을 활용한 텍스트 생성**에서는 요청하는 작업의 종류에 따라 최적의 프롬프트 작성 전략이 조금씩 다릅니다. 여기서는 정보 검색(Q&A), 코드 생성, 요약, 번역 등 대표적인 텍스트 기반 활용 분야별로 프롬프트 설계 팁을 정리합니다. 또한 각 작업을 넘어 공통으로 활용되는 고급 기법(Chain-of-Thought, Meta Prompting 등)도 함께 설명합니다.

### 정보 검색 및 질의응답 프롬프트

LLM을 질문 **answering**이나 정보 검색에 활용할 때는 정확성과 맥락 제공에 유의해야 합니다. 모델은 훈련된 지식으로 답변하지만 잘못된 정보(환각)를 그럴듯하게 만들어낼 수 있으므로 프롬프트 단계에서 이를 완화하는 전략이 필요합니다 .

- **질문의 명확화**: 찾고자 하는 정보가 무엇인지 구체적으로 물어보세요. 모호한 질문은 피하고, 가능하면 범위나 조건을 포함합니다 (예: "**18세기 유럽 역사에서 프랑스 혁명의 원인 3가지를 알려줘.**"). 필요한 경우 한 번에 하 나의 질문에 집중하고, 여러 질문은 나누어 묻는 것이 정확도를 높입니다.

- **배경지식 제공:** 모델이 알아야 할 전제가 있거나 참고할 자료가 있다면 프롬프트에 포함합니다. 예를 들어 "다음 글을 읽고 질문에 답해줘: [본문] ... 질문: ..."처럼 관련 텍스트를 첨부하거나, "네가 참고할 수 있는 데이터: ... 이제 이것 기반으로 답변해"라고 제시할 수 있습니다. 이렇게 하면 모델이 주어진 맥락 안에서만 추론하도록 유도할 수 있습니다.
- **체인-of-Thought 활용:** 복잡한 질문이나 논리적 추론이 필요한 경우 Chain-of-Thought(연쇄 사고) 기법을 사용합니다. 이는 모델에게 문제를 여러 단계로 나누어 생각하도록 유도하는 방법입니다. 예를 들어 "1단계: 문제를 정의한다. 2단계: 관련 정보를 식별한다. 3단계: 답을 도출한다. 위 단계를 따라 답변해줘."처럼 단계 별 지침을 프롬프트에 포함하면, 모델이 논리적 중간 과정을 거쳐 답을 도출하게 됩니다. 이런 단계적 사고 유도는 수학 문제 풀이, 복잡한 원인-결과 추론 등에 특히 효과적입니다. (단, 최종 답변에만 집중하고 중간 추론은 출력하지 않도록 하려면 "최종 답만 제시해."라고 마지막에 명시하세요.)
- **사실 검증 및 근거 요구:** 중요한 정보 검색에서는 모델 답변의 신뢰성을 높이기 위해, 프롬프트에서 근거 제시를 요구할 수 있습니다. 예를 들어 "근거가 되는 출처도 함께 알려줘."라고 하면 모델이 출력에 출처를 포함하거나, 최소한 사실 근거에 기반한 서술을 하도록 유도할 수 있습니다. (하지만 주의: 모델이 가짜 출처를 만들어낼 수도 있으므로, 중요한 용도의 경우 직접 검증이 필요합니다.) 대안으로, 최신 기법인 연쇄 검증(CoVe)을 적용하면 환각을 줄일 수 있는데, 이는 뒷부분 고급 기법 섹션에서 다루겠습니다.
- **응답의 형식 제어:** 질문에 대한 답변 형식을 지정하면 더 만족스러운 결과를 얻습니다. 예를 들어 "한 문장으로 대답해줘.", "표로 정리해줘.", "여러 후보를 번호 목록으로 제공해줘." 등으로 요청하세요. 특히 Q&A에서는 흔히 리스트 업이나 정의 + 예시 구조가 유용하므로, 원하는 구성을 미리 알려줄 수 있습니다.
- **모델 한계 감안:** 정보 검색 질문에서는 모델이 아는 범위를 넘어서면 그럴듯한 거짓을 말할 수 있다는 점을 명심해야 합니다. 프롬프트에 "모르는 내용은 모른다고 말해" 또는 "확실한 정보만 알려줘"라고 지시하면 어느 정도 환각을 억제할 수 있습니다. 그래도 중요한 사실은 사후에 따로 검증하는 것이 안전합니다.

## 코드 생성 프롬프트

프로그래밍 코드를 생성할 때의 프롬프트는 문제가 되는 상황을 정확히 기술하고, 원하는 출력의 조건을 명시하는 데 초점을 맞춥니다. 코드 생성용 모델은 문맥에 민감하므로, 다음과 같은 전략을 따르면 더 정확하고 실행 가능한 코드를 얻을 수 있습니다.

- **언어와 환경 명시:** 어떤 프로그래밍 언어의 코드를 원하는지 프롬프트에서 분명히 하세요 (예: Python, JavaScript 등). 필요하다면 해당 언어의 버전이나 사용해야 할 프레임워크/라이브러리도 지정합니다 (예: "Python 3.10 기준, pandas 라이브러리 사용"). 또한 함수인지 스크립트 전체인지, 클래스인지 등 출력 형태를 알려주세요. 예: "Java로 함수만 작성해줘.", "전체 실행 가능한 스크립트 형태로 보여줘." 등의 지시.
- **기능과 입력/출력 명세:** 만들고자 하는 코드의 기능 요구사항을 상세히 설명합니다. 예를 들어 "두 개의 정렬된 배열을 병합하는 함수를 작성해줘." 대신 "정수로 이루어진 두 개의 오름차순 정렬 리스트를 입력 받아, 하나의 오름차순 리스트로 병합하여 반환하는 Python 함수 merge\_sorted\_lists(list1, list2)를 작성해줘."처럼 함수명, 인자, 동작, 반환값까지 명확히 기술합니다. 추가로, 엠티

케이스나 성능 요구사항이 있다면 언급합니다 (예: 정렬 알고리즘에서 "시간 복잡도  $O(n)$  또는  $O(n \log n)$ 로 구현" 등).

- **예시와 경계 조건 제시:** 특히 복잡한 알고리즘이나 입출력 처리가 필요한 경우, 테스트 예시를 프롬프트에 포함 하면 도움이 됩니다. 예: "예시 입력: 리스트1=[1,3,5], 리스트2=[2,4,6]; 예시 출력: [1,2,3,4,5,6]". 이를 통해 모델이 정확한 동작을 이해하도록 합니다. 또한 "리스트가 빈 경우도 처리해야 함.", "음수 입력도 올 수 있음." 등 경계 조건을 미리 명시하여 코너 케이스 처리를 유도하세요.
- **코드 구조와 스타일 지침:** 원하는 코딩 스타일이나 제약이 있다면 함께 알려줍니다. 예를 들어 "재귀 호출은 피 하고 반복문으로 구현할 것.", "주석을 달아서 설명해줘.", "PEP8 스타일로 작성해줘." 등으로 요구할 수 있습니다. GPT-5 등 최신 모델은 구조화된 지침도 잘 따르므로, XML/JSON 형태로 규칙을 제공하는 것도 가능합니다. (예: 함수는 30줄 이내로 작성예외 처리를 포함 )
- **출력 형식 지정 (예: markdown 코드블록):** 프롬프트에서 "코드는 Markdown 형식의 코드 블록으로 출력해 줘."라고 하면 모델이 `python`으로 시작하는 코드블록에 답변을 넣어주어 복사하기 쉽게 만듭니다. 만약 코드 외에 설명도 원하면 "코드 뒤에 간략 설명도 덧붙여줘."처럼 요구사항을 분리해서 전달합니다. 반대로 코드 만 원하면 "불필요한 설명 없이 코드만 제시해."라고 못박아주세요.
- **단계별 접근:** 한 번에 복잡한 코드를 모두 생성하기어려울 경우 단계별 프롬프트로 접근합니다. 예를 들어 "1단계: 함수의 설계를 단계별로 생각해보고 의사코드(pseudocode)를 만들어줘." → (모델 응답 확인 후) "2단계: 이제 해당 의사코드를 Python 실제 코드로 구현해줘." 식으로 대화를 나누며 발전시킵니다. 이렇게 하면 모델 이 논리를 한번 검증한 후 코드를 작성하므로 오류 가능성을 낮출 수 있습니다.
- **문제 해결 및 디버깅 프롬프트:** 생성된 코드가 완벽하지 않을 수 있으므로, 디버깅에도 프롬프트를 활용합니다. 예를 들어 코드 실행 후 오류가 발생하면, 오류 메시지와 함께 "이 오류를 고쳐줘."라고 후속 프롬프트를 보냅니다. 모델은 오류 메시지를 분석해 코드를 수정하거나, 잘못된 부분을 설명하고 개선안을 제시할 수 있습니다. 이 처럼 코드 수정 루프를 통해 최종적인 동작 코드를 얻을 때까지 몇 차례 프롬프트-수정 작업을 수행할 수도 있습니다.
- **예시: 잘 작성된 코드 생성 프롬프트** "당신은 숙련된 파이썬 개발자입니다. 2개의 정렬된 리스트를 하나로 병합하는 함수 `merge_sorted_lists(list1, list2)` 를 구현하세요. `list1` 과 `list2` 는 오름차순으로 정렬된 정수 리스트이며, 함수는 새로운 오름차순 리스트를 반환해야 합니다. 단, 파이썬 내장 함수는 사용하지 말고 이중 반복문 없이  $O(n+m)$  복잡도로 구현하세요. 또한 예외 및 엣지 케이스를 처리하세요 (예: 한 쪽 리스트가 빈 경우 등). 함수 구현 후 간략히 동작을 설명하는 주석을 추가해 주세요. 코드 만 Markdown 형식으로 제공하고 추가 설명은 생략해주세요."

위 프롬프트는 언어(파이썬), 함수명과 기능, 성능 요구사항, 엣지 케이스, 스타일(주석 요구, 출력 형식) 등을 모두 포함 하여, 모델이 무엇을 어떻게 해야 할지 명확히 인식하도록 합니다. 이처럼 상세한 지시가 있을 때 모델은 요구에 부합하는 코드를 생성할 확률이 높아집니다 .

요약 프롬프트

문서나 대화의 요약을 요청할 때는 어떤 형태의 요약을 원하는지 명확히 전달하는 것이 중요합니다. LLM은 요약의 길이 나 상세 수준을 자유롭게 조절할 수 있으므로, 프롬프트를 통해 원하는 분량, 강조점, 형태를 지정해야 일치하는 결과를 얻을 수 있습니다.

- **요약 대상 제공**: 요약할 텍스트가 짧다면 프롬프트에 직접 포함하고, 길다면 핵심 부분만 추려서 제공하거나 모델이 접근할 수 있는 방식으로 전달합니다 (예: 시스템 메시지나 파일 형태 등). 모델이 해당 내용을 참고하여 작업하도록 "다음은 요약할 문서입니다: ..."와 같이 맥락을 분명히 합니다.
- **목적과 분량 지시**: 요약의 용도나 대상 독자를 알려주면 적합한 수준으로 요약됩니다. 예를 들어 "비전공자용으로 쉽게", "경영진 보고용 한 장 요약" 등 맥락을 주십시오. 또한 "한 문단 요약", "3줄로 요약", "200자 이내로 요약"처럼 길이 제한을 명시하면 과도하게 장황한 답을 피할 수 있습니다. 구체적인 문장 수나 글자 수, 또는 "간략히" vs "상세히" 같은 표현으로 분량을 제어합니다.
- **중점 사항 강조**: 원문에서 어떤 점을 중심으로 요약해야 하는지 지시할 수 있습니다. 예를 들어 "주요 등장인물과 결말에 초점을 맞춰 소설 줄거리를 요약해줘.", "논문의 연구목적과 결론에 중점을 두어 요약해줘."처럼 원하는 핵심 포인트를 미리 알려줍니다. 반대로 "기술 세부사항은 제외하고"처럼 제외할 요소도 지정 가능합니다. 이렇게 하면 모델이 요약 시 중요도를 판단하는 데 도움이 됩니다.
- **형식 지정**: 요약 결과를 **\*\* bullet** 포인트 목록, 숫자 목록, 표, 문단 **\*\*** 등 원하는 형식으로 요청하세요. 예를 들어 "회의 요약의 항목 목록으로 만들어줘. 각 항목은 회의에서 결정된 사항으로 해줘.", "챕터별로 한 줄 요약해줘." 등으로 출력 구조를 지정하면 체계적인 결과를 얻을 수 있습니다. 또한 필요하다면 "각 요약 앞에 챕터 번호를 붙여줘." 같이 세부 형식도 설정합니다.
- **원문 톤 반영 여부**: 요약 시 원문의 어조나 전문성 수준을 유지할지, 아니면 새롭게 바꿀지도 결정합니다. 예를 들어 "이메일 내용을 공손한 비즈니스 어투로 요약해줘.", "원문이 캐주얼하지만 공식적인 톤으로 다시 써줘."처럼 요구할 수 있습니다. 대부분의 경우 요약에서는 간결하고 객관적인 어조가 좋지만, 상황에 따라 조정 가능합니다.
- **검증과 편집**: 모델이 제시한 요약이 의도와 다르다면 피드백을 주고 후속 프롬프트로 개선할 수 있습니다. "너무 간략해. 세부 내용도 조금 포함해서 다시 써줘."처럼 추가 지시를 내리면 모델이 이전 답변을 참고해 더 원하는 쪽으로 수정합니다. 특히 긴 문서를 다룰 때는 한 번에 요약이 잘 안 될 수 있으므로, 필요한 경우 부분별로 요약 후 통합하거나, "1차 요약 -> 보충 지시 -> 최종 요약"의 단계를 거치는 것이 좋습니다.

## 번역 프롬프트

LLM을 번역에 활용할 때는 단순히 문장을 넣는 것보다 번역 방향과 스타일을 구체화하는 프롬프트가 더 나은 결과를 제공합니다. 모델은 맥락과 뉘앙스를 고려해 번역하지만, 프롬프트를 통해 원하는 언어적 톤과 형식을 명확히 전달해야 합니다.

- **언어와 방향 명시**: 어떤 언어로 어디에서 어디로 번역해야 하는지 분명히 씁니다. 예: "한국어를 영어로 번역해줘.", "영어 문장을 한국어 격식체로 번역해줘.". 특히 다의어가 많은 언어쌍에서는 이 지시가 중요합니다. 또한 영어 같은 경우 미국식 vs 영국식 등의 지역 표현 차이가 중요하면 언급하세요.



- **존칭/어투 지정**: 번역의 말투(tone)와 존댓말/반말 여부를 지시합니다. 한국어 번역의 경우 "반말로", "격식을 차려", "존댓말로 부드럽게" 등 어투를 미리 알려주면 일관성 있는 번역이 됩니다. 예를 들어 "이 문장을 예의를 갖춘 비즈니스 한국어로 번역해줘."라고 하면 "...드립니다."와 같은 어미를 사용하게 유도할 수 있습니다.
- **용어와 이름 처리**: 전문 용어나 고유명사는 번역에서 특별히 다룰 수 있습니다. 번역하지 않을 용어는 따로 알려 주세요 (예: "'OpenAI' 같은 고유명사는 번역하지 말 것"). 반대로, 꼭 번역해야 하는 용어가 있다면 해당 번역어를 제시해 주는 것도 방법입니다. 또한 측면 주석으로 "(주: 000는 독일의 지명입니다)"처럼 맥락 정보를 제공하면 정확한 고유명사 번역에 도움이 됩니다.
- **형식과 부가 요소 유지**: 원문에 목록, 문단, 줄바꿈 등이 있다면 번역 결과에도 유지하라고 지시할 수 있습니다. 예: "원문의 줄바꿈과 목록 형식을 그대로 유지하면서 번역해줘.". 또한 코드 스니펫이나 수식이 섞여 있다면 "코드 부분은 번역하지 말고 그대로 둘 것"처럼 특정 부분을 예외로 지정합니다. 이런 디테일을 미리 언급하면 후처리 작업을 줄일 수 있습니다.
- **의역 vs 직역 수준**: 번역이 얼마나 의역(자연스러운 표현) 또는 직역(원문 충실)이어야 하는지도 요구할 수 있습니다. "가능한 한 직역해줘." vs "매우 자연스럽게 의역해줘." 등으로 모델의 번역 스타일을 통제합니다. 예를 들어 문학 작품이라면 의역을, 법률문서라면 직역을 선호하는 식입니다. 또한 "중요한 뉘앙스나 강조점은 살려 줘."라고 요청하면 모델이 원문의 강조를 유지하려 시도할 것입니다.
- **번역 예시 제공**: 어려운 한문장 번역 등에는 원하는 번역 품질을 가능할 수 있는 짧은 예시를 함께 주는 것도 가능합니다. 예를 들어 첫 문장을 직접 번역해서 보여주고 "나머지도 이런 톤으로 번역해줘."라고 하면 모델이 그 스타일을 이어받습니다. 다만 지나친 예시 제공은 모델이 그대로 베끼려 할 수 있으니 필요한 최소한으로만 합니다.
- **예시**: 좋은 번역 프롬프트 "다음은 격식 있는 서울 표준말로 번역해 주세요. 전문 용어는 가능한 한 직역하되, 자연스러운 흐름을 유지합니다. 필요한 경우 문장을 나눠도 좋습니다. 원문: **We are thrilled to announce the launch of our new AI platform. It's a revolutionary step forward in machine learning and data analysis.**"

위 프롬프트에서는 목표 언어와 어투(격식 있는 서울 표준말)를 지정하고, 전문 용어는 직역하지만 자연스러운 흐름도 요구하여 균형을 잡았습니다. 또한 한 문장이 너무 길 경우 나눠도 된다는 재량도 주었습니다. 이렇게 번역의 기준을 프롬프트에 명시하면 모델이 임의로 판단하는 것을 줄이고 사용자의 의도에 맞는 번역을 제공합니다. 참고로 IBM의 예시에서도 "Translate this text into French: 'Hello.'"처럼 언어와 내용을 명확히 제시하는 것이 기본임을 보여줍니다.

**최신 고급 기법: 메타 프롬프팅, 자기성찰, ToT, CoVe 등**

위에서 다룬 원칙과 전략 외에도, 2023년 이후 연구에서 제안되어 LLM의 성능을 향상시키는 몇 가지 고급 프롬프트 기법들이 있습니다. 여기서는 그중 실무에 바로 적용할 수 있는 대표 기법 4가지를 소개합니다. 이러한 방법들은 복잡한 문제 해결, 사실 검증, 출력 품질 개선 등에 효과가 있다고 보고되고 있습니다.

**메타 프롬프팅 (Meta Prompting)**

**메타 프롬프팅**이란 **LLM** 자체를 활용하여 더 좋은 프롬프트를 생성하거나 기존 프롬프트를 개선하는 기법입니다. 일반적인 프롬프트 엔지니어링이 사람이 직접 프롬프트를 만드는 것이라면, 메타 프롬프팅은 모델에게 "스스로에게 줄 프롬프트를 만들어봐"라고 요청하는 셈입니다. 이를 통해 모델이 구조적으로 자신의 지시를 최적화하거나 다른 시각에서 문제를 접근하게 할 수 있습니다.

- **방법**: 가장 간단한 형태의 메타 프롬프트는 "스스로에게 가장 도움이 될만한 질문을 생성하고, 그에 답해"라는 식입니다. 예를 들어 사용자가 원하는 작업이 "기후 변화 개념 설명"이라면, 메타 프롬프트로 모델에게 "기후 변화를 쉽게 설명하기 위한 가장 좋은 프롬프트를 만들어줘"라고 먼저 지시할 수 있습니다. 모델이 생성한 프롬프트(예: "기후 변화, 원인과 영향에 대해 어린이가 이해할 수 있도록 간단히 설명하라")를 다시 모델에 입력해 답을 얻는 것입니다. 이 과정을 통하면 초기 막연한 요청을 모델 스스로 구체화하여 더 나은 출력을 이끌어낼 수 있습니다.
- **변형**: 좀 더 복잡한 메타 프롬프팅에는 여러 프롬프트 후보를 생성하고, 각각 출력해본 뒤 성능 평가를 통해 최적 프롬프트를 선택하는 방법도 있습니다. **Automatic Prompt Engineer** 등의 기법은 모델이 여러 프롬프트를 시도한 후 스스로 평가하여 최고 점수의 프롬프트를 선택하도록 합니다. 또한 **Prompt Generator**라 불리는 도구/모델을 활용해 인간 대신 프롬프트 초안을 얻고, 이를 토대로 최종 프롬프트를 다듬는 방법도 있습니다. **Anthropic**이나 **OpenAI**에서는 개발자 콘솔에 이런 프롬프트 생성 보조 기능을 제공하기도 합니다.
- **활용 예**: 메타 프롬프팅은 특히 복잡한 과제 초기 설계 단계에서 유용합니다. 예를 들어 "신규 사용자 온보딩용 챗봇 스크립트를 작성"해야 한다면, 바로 시작하지 않고 모델에게 "좋은 온보딩 챗봇 대본 작성을 위한 프롬프트 지침"을 만들어보라고 할 수 있습니다. 모델은 필요한 요소들을 나열한 프롬프트 초안을 제시할 것이고, 이를 토대로 실제 콘텐츠 생성을 수행하면 더 구조화된 결과를 얻을 수 있습니다. 또한 여러 분야 전문가 캐릭터(**LM**들)를 두고 하나의 컨트롤러 **LLM**이 이들을 조율하는 메타 프롬프팅 방식도 연구되고 있습니다.

요약하면, 메타 프롬프팅은 "프롬프트를 작성하는 프롬프트"로서, **LLM**의 힘을 프롬프트 설계에 역으로 활용하는 접근입니다. 이를 통해 프롬프트 개발 시간이 단축되고 예상치 못한 각도의 아이디어를 얻는 등 이점이 있습니다. 다만 최종 출력의 품질은 생성된 프롬프트의 품질에 좌우되므로, 인간이 그 과정을 검토하며 사용하는 것이 안전합니다.

## 자기성찰 (Self-Reflection)

**자기성찰(Self-Reflection) 기법**은 모델에게 자신의 답변이나 추론 과정을 되돌아보고 오류를 찾아 개선하도록 유도하는 방법입니다. 이는 일종의 자기 비평 단계를 거치는 것으로, 사람으로 치면 답을 쓴 뒤 검토하면서 잘못된 부분을 수정하는 행위와 유사합니다. 자기성찰을 통해 **LLM**이 처음에는 놓쳤던 논리적 오류나 사실 오류를 발견하고 고칠 수 있다는 연구 결과들이 나오고 있습니다.

- **방법**: 가장 단순한 형태는 2단계로 이루어집니다. ① 초안 응답을 생성한 후, ② 그 초안에 대해 모델 스스로 비판/검토하도록 프롬프트를 줍니다. 예를 들어, 사용자가



수학 문제를 풀게 한 경우 1차 답변 후에 "네 답이 틀렸을 가능성이 있다면 그 이유를 설명하고 정답을 다시 시도해봐."라고 지시할 수 있습니다. 또는 "위 답변에서 논리적으로 모순되거나 증거가 부족한 부분을 찾아 수정해."와 같은 프롬프트를 후속으로 보냅니다. 모델은 자신의 이전 응답을 연쇄 사고(CoT)로 분석하며 오류 원인을 설명하고, 개선된 답을 제시하게 됩니다 .

- **한번에 자기성찰 유도**: 대화 턴을 늘리지 않고 한 번의 프롬프트 내에서 자기성찰+수정을 시킬 수도 있습니다. 예를 들어 프롬프트에 "먼저 질문에 대한 답을 제시한 뒤, 그 답이 타당한지 자체 평가하고 필요하다면 답을 수정하세요."라는 지침을 넣는 것입니다. 모델은 답변을 하고 이어서 자기평가를 수행한 뒤 최종 수정을 하는 형태로 응답할 수 있습니다. 다만 모든 모델이 이 복합 지시를 충실히 수행하는 것은 아니므로, 잘 안 될 경우 단계별로 나누는 것이 확실합니다.
- **효과**: 자기성찰 단계를 거치면 모델의 답변 정확도가 향상되고 편향이나 부적절함이 줄어든다는 보고가 있습니다 . 실제 한 연구에서는 유해한 발언이 75% 감소하고 비독성 출력은 거의 유지되었다고 합니다 . 또 다른 연구에서는 수학 문제 등 문제 해결에서 자기성찰을 넣었더니 정답률이 높아졌다고 합니다 . 이는 모델이 자신의 Chain-of-Thought를 검토하고 오류를 식별할 수 있기 때문으로, “모델이 스스로 학습한다”는 관점에서 흥미로운 진전입니다. 무엇보다도, 이런 개선이 별도 파인튜닝 없이 추론 시간에 간단한 프롬프트 추가만으로 이뤄진다는 점이 실용적 이점입니다.
- **활용 예**: 자기성찰 기법은 코드 생성 후 디버깅, 복잡한 분석 결과 검토, 창작 글 다듬기 등 다방면에 활용 가능합니다. 예를 들어 모델이 작성한 코드가 예러가 난 경우 "이 코드의 문제점을 스스로 찾아 수정해봐."라고 시키거나, 모델이 쓴 글 초안에 대해 "자연스러운 흐름인지 스스로 점검하고 개선 제안해봐."라고 할 수 있습니다. 모델은 이전 출력에 대한 메타인지적 평가를 수행하고 더 나은 결과를 제시합니다. 또한 Anthropic의 헌법 AI에 서는 답변 생성 후 헌법 원칙에 비추어 자기비평을 하는 과정이 포함되는데, 이것도 자기성찰의 한 사례라 볼 수 있습니다.

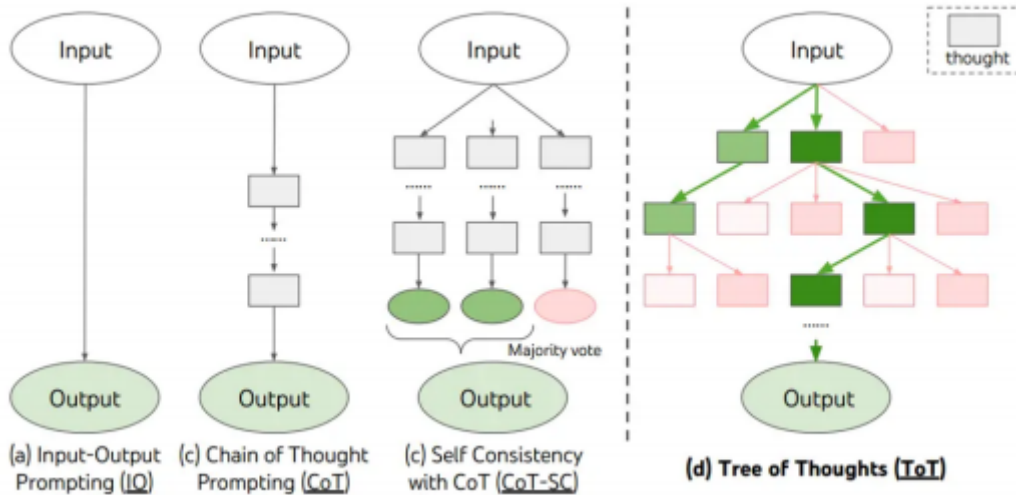
정리하면, 자기성찰 프롬프팅은 모델에게 “네 스스로 너의 답을 검토해봐”라고 묻는 것으로, 올바른 피드백만 주어진다면 모델은 스스로 오류를 교정하고 답을 개선할 수 있습니다 . 물론 모델이 잘못된 판단을 내릴 수도 있으므로, 중요 의사결정에서는 여전히 최종 인간 검토가 필요합니다. 하지만 일반적인 사용 맥락에서 자기성찰을 포함시키면 LLM 활용의 안정성과 품질을 한 단계 높일 수 있는 것으로 평가됩니다.

## 사고의 나무 (Tree-of-Thoughts, ToT)

**사고의 나무(Tree-of-Thoughts, ToT) 기법**은 복잡한 문제를 풀 때 모델이 하나의 직선적인 사고(chain-of-thought) 대신 여러 갈래의 생각을 탐색하도록 유도하는 접근입니다. 2023년 Yao 등 연구자들이 제안한 이 방법은, 마치 체스에서 다양한 수를 탐색하듯이 LLM이 여러 중간 단계 결과(생각)를 나무 구조로 전개하고 평가하며 최선의 답을 찾아가도록 합니다 .

- **개념**: ToT에서는 생각(thought)을 문제 해결의 한 단계를 기술하는 문장이나 표현으로 정의합니다. 모델은 처음에 여러 가지 가능한 생각(중간 단계 해법)을

생성하고, 각 생각에서 다시 다음 단계로 전개되는 가지(branch)를 펼칩니다. 이렇게 생성된 생각의 나무에서 가지치기를 하거나 더 탐색할 가지를 선택하는 탐색 알고리즘(DFS, BFS 등)을 적용하여, 가장 유망한 해결 경로를 찾아 최종 답을 구성합니다 .



**Figure: Tree-of-Thoughts** 개념도.

(단일 연쇄 사고(CoT) 방식(a)과 달리, ToT는 여러 경로를 동시에 고려하며(b)(c) 탐색 알고리즘을 통해 최적 경로를 선택한다. 연구 결과 ToT는 기존 기법 대비 문제 해결 성능을 크게 향상시켰다.)

- **프롬프트 구현**: ToT 원리를 프롬프트 수준에서 간단히 적용하려면, 모델에게 "여러 가능성을 탐색해보고 판단하라"는 지침을 주면 됩니다. 예를 들어 수수께끼를 풀 때 "가능한 해석 A, B, C를 각각 생각해보고, 그 중 가장 그럴듯한 답을 선택해줘."라고 할 수 있습니다. 또는 "1단계에서 여러 해법 아이디어를 제안하고, 2단계에서 각 아이디어의 결과를 평가한 후, 최종 해법을 결정해."처럼 명시적으로 단계 구조를 지정할 수도 있습니다. **Hulbert(2023)**의 연구에서는 한 프롬프트 안에서 "여러 중간 생각을 생성하고 각각 '확실/애매/불가능' 여부를 평가하라"는 형식으로 ToT를 구현하기도 했습니다 .
- **효과와 활용 분야**: ToT는 복잡한 퍼즐, 경로 찾기, 수학적 최적화, 논증 전개 등 정답을 탐색해야 하는 문제에 유용합니다 . 예컨대 논리 퍼즐에서 한 가지 가정을 따라가 보면 모순이 생길 수 있는데, ToT는 동시에 여러 가정을 분기 탐색해 가장 일관된 해를 찾도록 합니다. 실제 실험에서 ToT는 기존 **Chain-of-Thought**보다 상당한 성능 향상을 보였으며, 일부 영역에서는 정답률을 두 배 이상 높였습니다 . 이는 나무 탐색을 통해 잘못된 방향으로 깊이 파고드는 일을 줄이고, 보다 전방위적인 탐색을 가능하게 했기 때문입니다.
- **한계**: ToT의 완전한 구현은 여러 차례의 모델 호출과 복잡한 컨트롤 알고리즘을 필요로 하기 때문에, 일반적인 사용자 프롬프트에 직접 적용하기엔 제한이 있습니다 . 다만 간소화된 형태로, 예를 들어 "여러 해결책을 생각한 후 최선의 것을 골라 답해"라는 프롬프트는 충분히 적용 가능합니다. 또한 **MultiAgent** 토의 형태로 각 에이전트가 다른 가정을 탐색하게 한 뒤 취합하는 등 변형 기법도 고안되고 있습니다.

요약하면, 사고의 나무(**ToT**)는 **LLM**에게 “여러 갈래로 생각해보고 판단하라”고 가르치는 접근입니다. 이를 통해 한 방향으로 실수할 확률을 줄이고 보다 종합적으로 사고하게 하여, 난해한 문제에서 기존 프롬프트 기법 대비 우수한 성과를 낼 수 있음이 입증되고 있습니다. 향후 **RL** 등을 결합해 모델이 스스로 가지치기 전략을 학습하도록 발전하는 중이며, 복잡한 의사결정 시뮬레이션 등에 적용 가능성이 기대됩니다.

## 연쇄 검증 (**Chain-of-Verification, CoVe**)

**연쇄 검증(CoVe) 기법**은 **LLM**의 환각(**hallucination**) 문제, 즉 그럴듯하지만 잘못된 정보 생성을 줄이기 위해 고안된 프롬프트 전략입니다. 2023년 **Meta AI** 연구진(**Dhuliawala** 등)이 발표한 방법으로, 모델에게 답변을 바로 주는 대신 자신의 답을 검증하는 질문들을 만들고 확인하도록 유도합니다. 일종의 **QA** 모드로 자기 자신을 교차검증하는 셈입니다.

- **절차**: **CoVe**는 4단계 프롬프트 체인으로 이루어집니다. 각 단계는 별도의 프롬프트-응답으로 진행됩니다 (동일한 모델 인스턴스를 4번 호출하거나, 대화에서 4턴으로 나뉘도 무방). 단계는 다음과 같습니다.
- **기본 답변 생성**: 먼저 사용자의 질문에 대한 초기 답변 초안을 얻습니다. (예: "Q: ...? A: ..." 형태)
- **검증 질문 생성**: 이어서 모델에게 "위 답변의 각 부분이 맞는지 확인하기 위한 질문들을 생성"하도록 프롬프트를 줍니다. 즉, 사실 검증을 위한 소질문 리스트를 뽑아내는 단계입니다. (예: "해당 답변의 진위를 확인하기 위한 질문 3가지를 만들어줘.")
- **검증 질문에 답하기**: 두 번째 단계에서 나온 각 검증 질문에 대한 답변을 모델에게 구하도록 합니다. (예: "Q1: ...? A1: ...; Q2: ...? A2: ..." 등)
- **최종 답변 조정**: 마지막으로 모델에게 "초기 답변과 검증 결과를 종합하여, 틀린 정보를 제거하거나 수정한 최종 답변을 작성"하게 합니다. 이때 이전 단계에서 얻은 검증 답변들을 참고하도록 프롬프트에 포함시킵니다. (예: "위 검증 답변들을 반영해, 최초 답변을 정확하게 수정해줘.")

이러한 일련의 과정을 통해, 모델은 처음에 생성한 답변을 스스로 교차검증한 뒤 오류를 걸러낸 최종 답을 내놓게 됩니다.

- **예시**: "뉴욕 출생의 영화감독은 누구인가?"라는 질문에 모델이 처음 **Spike Lee**를 뉴욕 출생이라고 잘못 답했다고 가정합니다. **CoVe**를 적용하면 2단계에서 "**Spike Lee**의 출생지를 확인하는 질문" 등이 생성되고, 3단계에서 "**Spike Lee**는 조지아주 애틀랜타 출생"이라는 검증 답이 나오므로, 4단계 최종 답변에서는 **Spike Lee**를 제거하고 정확한 감독들만 나열하는 식입니다. 이처럼 모델이 짧은 검증 Q&A들을 통해 스스로 오류를 걸러내도록 하는 것이 핵심입니다.
- **효과**: **CoVe**는 다양한 지식 태스크에서 환각 감소에 큰 효과를 보였습니다. **Meta**의 실험에 따르면, **Wikidata** 기반 질의응답, 위키 문서 기반 리스트업, **MultiSpanQA**,

인물 전기 생성 등 4가지 과제 전반에서 기존 Fewshot 및 Chain-of-Thought 기법 대비 23%~100% 이상 정확도가 향상되었습니다. 특히 사실 정확도가 필요한 작업에서 정보 손실 없이 정밀도를 높였다는 평가입니다. 이런 성과 때문에 CoVe를 LLM 정보 생성 파이프라인의 기본 단계로 넣는 것도 고려할 만하다고 할 정도입니다.

- **유의사항:** CoVe의 단점은 한 질문에 대해 4배 많은 프롬프트 호출이 필요하다는 점입니다. 따라서 API 비용이나 지연시간이 증가할 수 있습니다. 또한 검증 질문 자체에 대한 답변이 틀릴 경우 그 한계도 있습니다. 그럼에도, 최신 거대모델에서는 짧은 질답은 비교적 정확히 수행하므로 전체적인 개선 효과가 이 단점을 상회하는 것으로 보입니다. 만약 리소스가 허용된다면, 중요한 질의에 CoVe를 적용해 안전장치로 삼을 수 있습니다. 실무적으로는, 한 번에 모두 자동화하지 않고 2단계나 3단계 정도를 인간 검토와 혼합하여 구현하는 타협안도 고려됩니다.

요약하자면, 연쇄 검증(CoVe)은 "먼저 답하고, 스스로 질문하고, 검증하고, 수정해라"라는 4단계 사고 루프를 모델에게 부여하는 기법입니다. 이를 통해 사실적 정확도를 크게 끌어올릴 수 있으나, 실행 비용이 늘어나는 단점이 있습니다. 그러나 신뢰성이 중시되는 보고서 작성, 지식 그래프 구축 등의 작업에서 CoVe는 매우 유용한 도구이며, 최신 프롬프트 엔지니어링의 중요한 발전으로 평가받고 있습니다

#### 4. 이미지 생성 프롬프팅 전략: 효과적인 이미지 프롬프트 구성법

텍스트-투-이미지 모델(예: DALL·E 3, Google Imagen, Midjourney, Stable Diffusion 등)에서 원하는 이미지를 얻으려면, 프롬프트에 시각적 장면에 대한 상세한 묘사를 포함해야 합니다. 텍스트만으로 이미지를 묘사하는 것이므로, 무엇을 그리고 어떤 스타일로 표현할지를 명확히 전달하는 것이 핵심입니다. 최신 이미지 생성 모델들의 공식 가이드와 노하우를 토대로 효율적인 프롬프트 구성 요령을 정리하면 다음과 같습니다.



Style Subject  
A sketch of a modern apartment building  
surrounded by skyscrapers  
Context and Background

예시 이미지 프롬프트: "현대식 아파트 건물(주제)"이 "마천루로 둘러싸인(배경)" 장면을 "스케치 스타일(스타일)"로 묘사한 프롬프트의 생성 결과. 이처럼 주제(subject),

배경/맥락(context), 스타일(style) 세 요소를 명확히 포함하면 원하는 이미지에 가까워집니다

- **주제 (Subject) 명시:** 프롬프트의 첫 부분에는 그리고자 하는 대상을 정확히 적어주세요. 사람, 동물, 사물, 풍경 등 무엇이 중심인지 분명해야 모델이 혼동하지 않습니다. 예를 들어 "a red sports car" 또는 "an elderlyman with a cane"처럼 주어 + 핵심 속성 형태로 간결하게 제시합니다. 주제가 복수라면 함께 언급하되, "1명의 소녀와 1마리 고양이"처럼 수량도 명시하면 좋습니다. 주제가 없거나 애매하면 모델 출력이 산만해질 수 있으므로 프롬프트의 시작은 항상 "무엇의 그림인지"를 포함하세요.
- **배경과 맥락 (Context & Background) 추가:** 주제를 어떤 환경, 상황, 배경에 놓을지도 기술합니다. 예를 들어 "sitting on a beach at sunset" 또는 "in a bustling city street"처럼 배경 장소와 분위기를 그려주세요. 배경을 지정함으로써 이미지의 스토리가 생기고, 모델이 시각적 배치를 더 잘 구성합니다. 맥락에는 장소뿐 아니라 행동이나 상황도 포함될 수 있습니다 (예: "a group of friends celebrating a birthday in a park"). 다만 너무 많은 요소를 한 프롬프트에 넣으면 산만해질 수 있으니, 핵심 배경 1~2개 정도로 제한하는 것이 좋습니다.
- **스타일 (Style)과 형식 지정:** 생성될 이미지의 시각적 스타일이나 질감을 분명히 요청하세요. 회화, 사진, 만화, 3D 렌더링 등 원하는 스타일을 나열합니다. 예를 들어 "in a watercolor painting style", "as a Pixarstyle 3D animation", "like an old black-and-white photograph" 등으로 가능합니다. 또한 카메라 시점, 화질, 색감도 조정할 수 있습니다. "close-up portrait, 85mm lens, bokeh background"처럼 카메라 설정을 넣으면 사진풍 묘사에 도움이 됩니다. "soft diffused lighting, golden hour colors"처럼 조명과 색조를 언급하면 분위기를 한층 정확히 전달할 수 있습니다. 스타일 관련 키워드는 여러 개를 나열해도 비교적 잘 반영되지만, 서로 어울리도록 조합하는 것이 중요합니다 (예: "낮과 밤" 같이 상충되는 요소는 피함).
- **부정 프롬프트 활용 (원하지 않는 요소 제외):** 이미지 프롬프트에는 포함하고 싶지 않은 요소를 지정할 수도 있습니다. Midjourney의 --no 나 Stable Diffusion의 negative prompt 필드를 활용하여, 예를 들어 "--no text, watermark, people"처럼 글자나 워터마크, 불필요한 인물 등이 나오지 말 것을 지시할 수 있습니다. 흔히 품질 향상을 위해 "no blur, no low-resolution, no artifacts" 등을 부정 프롬프트에 넣기도 합니다. 이를 통해 모델이 특정 방면으로 과도하게 출력하는 것을 억제하고 원치 않는 왜곡(특히 사람 손가락 등 잘못 그리기 쉬운 부분)을 줄일 수 있습니다.
- **구도와 세부 묘사:** 이미지의 구도(composition)를 결정하는 키워드도 효과적입니다. "wide angle shot", "bird's-eye view", "rule of thirds composition" 같이 촬영 기법 용어나 구도 원칙을 넣으면 결과에 반영됩니다. 또한 그림의 세부 요소들을 함께 묘사하여 풍부하게 만들 수 있습니다. 예를 들어 "ancient library interior, scattered old books, rays of light through dusty air"처럼 주요 주제 외에도 주변 디테일을 간결히 덧붙입니다. 이때도 콤마(,)로 구분해 나열하면 모델이 각각을 개별 특징으로 인식합니다. 다만 너무 길게 나열하면 어떤 요소는 무시될 수 있으니, 중요한 키워드를 앞부분에 배치하고 3~7개의 핵심 어구로 요약하는 것이 좋습니다.
- **명료하고 간결한 언어:** 이미지 프롬프트는 간결한 키워드의 나열로 이루어지는 경우가 많습니다. 완전한 문장보다는 키워드, 구 동사들의 나열이 모델에 유리한 경우도 있습니다. 예를 들어 "futuristic city skyline, neon lights, rainy night, cinematic"처럼 콤마로 구분된 어구들이 흔히 쓰입니다. Midjourney 가이드에 따르면 짧고 간단한 프롬프트가 오히려 더 좋은 이미지를 낼 때도 많다고 합니다. 따라서

불필요한 수식어나 접속사는 빼고 핵심 명사와 형용사 위주로 구성하세요. 영어를 사용하는 모델이 대부분이므로, 영어로 작성하는 것이 가장 좋습니다 (한국어도 지원되지만 일부 모델은 영어 prompt에서 최적 성능을 냅니다). 그리고 철자가 틀리거나 하면 엉뚱한 해석을 할 수 있으니 맞춤법과 표기도 신경쓰세요.

- **반복과 시행착오 (Iteration)**: 이미지 생성을 프롬프트 한 번에 끝내기보다, 여러 번 시도하며 개선하는 것이 일반적입니다. 먼저 핵심 아이디어만 넣은 짧은 프롬프트로 시도해보고, 결과에서 부족한 점을 파악하여 프롬프트를 조금씩 수정합니다. 예를 들어 첫 결과가 마음에 들지만 색감이 아쉬우면 "vivid colors"를 추가하고, 구도가 아쉽다면 "wide shot" 등을 추가하는 식으로 한 번에 한 요소씩 변경하며 비교해보세요. 이렇게 하면 무엇이 어떤 영향을 주는지 감을 잡을 수 있고 최적의 조합을 찾아갈 수 있습니다. 각 모델이 선호하는 프롬프트 스타일이 다르므로, 가이드라인을 참고하되 직접 실험을 통한 보정이 최선입니다.
- **모델/플랫폼별 팁**: DALL·E나 Imagen은 비교적 직관적으로 문장을 해석하는 편이고, Midjourney나 Stable Diffusion은 스타일 지시어에 민감합니다. Midjourney의 경우 --ar 16:9 (화면비), --s 750 (스타일 강도) 등의 매개변수를 프롬프트 끝에 사용할 수 있고, Stable Diffusion 계열은 ((( ( ... )))나 :1.5 같은 가중치 구문으로 특정 요소 비중을 높일 수 있습니다. 사용하는 툴의 매뉴얼을 참조하여 이러한 전용 기능도 활용하면 좋습니다. 예컨대 Midjourney에서는 --stylize 값으로 화풍 적용 강도를 조절하거나, 업스케일/버전 옵션으로 세부 품질을 높일 수 있습니다.
- **예시: 잘 정돈된 이미지 프롬프트 예** **fantasy ranger character, leather cloak, ash-blond hair, green eyes, woodland setting; style: cinematic 4K portrait, shallow depth of field; lighting: soft rim light, cool ambient; --no watermark, text** – 이 프롬프트는 주제(판타지 레인저 캐릭터 + 외모 특징), 배경(숲), 스타일(시네마틱 4K 인물사진), 조명(부드러운 테두리 조명) 등을 세미콜론이나 콤마로 구분해 명료히 기술하고, 원치 않는 요소(워터마크, 텍스트)는 --no 로 제외했습니다. 이렇게 작성하면 각 요소가 이미지에 잘 반영되어, 예를 들어 밝은 가장빛이 도는 선명한 숲속 인물 사진이 생성됩니다.

마지막으로, 이미지 모델별로 프롬프트 해석 특성이 다르므로, 하나의 프롬프트를 여러 모델에 시도해보며 가장 마음에 드는 출력을 주는 모델을 선택하는 것도 방법입니다. 예술적 스타일의 Midjourney, 사실적 묘사의 DALL·E, 사용자 세부제어가 강점인 Stable Diffusion 등 장단점이 있으니, 목적에 따라 플랫폼을 선정하고 거기에 맞게 프롬프트를 튜닝하세요.

## 5. 영상 생성 프롬프팅 전략: 텍스트-투-비디오 프롬프트 작성법

2025년 현재 텍스트로 영상을 생성해주는 모델과 서비스들이 등장하여 발전하고 있습니다. OpenAI의 Sora 2, PikaLabs, Runway Gen-2, Google의 Phenaki 계열 등이 대표적이며, 사용자는 짧은 프롬프트로 동영상 클립을 만들어낼 수 있습니다. 영상 프롬프트는 이미지 프롬프트와 유사하면서도, 시간 축을 고려한 움직임과 장면 묘사가 추가로 필요합니다. 다음은 영상 생성에 효과적인 프롬프트 구성 전략입니다.

- **시각 요소 + 동적 요소 모두 기술**: 텍스트-투-비디오 프롬프트에는 눈에 보이는 정적인 요소(인물/사물/배경)와 시간에 따른 동적 요소(동작/이동/카메라 움직임)가 모두 포함되어야 합니다. 예를 들어 "A raccoon in a plain room (시각적 설정: 라쿤과 방) trying to steal garbage, which floats in zero gravity (동적 내용: 쓰레기가 무중력



상태로 떠다니고, 라쿤이 훑치려 함)"처럼, 무엇이 보이고 어떤 움직임이 일어나는 지 한 문장에 묘사합니다. 이렇게 정적 묘사 + 동적 행동\*을 모두 명시하면 모델이 영상 장면을 보다 정확히 렌더링 할 수 있습니다.

- **카메라 구도와 움직임 지정**: 영상은 카메라 워크에 따라 느낌이 크게 달라지므로, 어떤 카메라 시점/이동으로 촬영된 장면인지를 프롬프트에 넣는 것이 좋습니다. 예를 들어 "A handheld low-angle tracking shot of an astronaut skateboarding on the moon"처럼 촬영 기법(손으로 든 듯한 저각 추적샷)을 언급할 수 있습니다. 또한 "smooth pan to the right", "rapid zoom-in on the face" 등 카메라 움직임을 직접 기술하면 (비록 미세 조정은 어려워도) 모델이 방향을 잡는 데 도움이 됩니다. Runway 연구에 따르면 프롬프트 구조를 "[카메라] 쏘트로 [주체]가 [동작]한다, ..." (뒷부분 부가설명)" 형식으로 짜는 것을 추천하고 있습니다. 이러한 구조를 따르면 프롬프트를 체계적으로 관리하기에도 용이합니다.
- **동작의 순서와 속도 표현**: 영상에는 시간이 흐르며 발생하는 사건의 순서가 중요합니다. 한 프롬프트 안에서 여러 장면을 표현하려면 문장을 나누거나 접속사를 써서 순서를 나타낼 수 있습니다 (예: "A man enters a room, then sits at a desk and starts typing."). 또한 "slowly", "suddenly", "in fast motion" 등의 부사나 표현으로 속도/타이밍을 전달하세요. 예를 들어 "runs in slow motion"이면 슬로모션 효과처럼 이해할 수 있습니다. 다만 현 기술상 텍스트만으로 아주 정확한 타이밍 조절은 어렵지만, 시도해볼 가치는 있습니다. Runway Gen-2의 고급 기능으로 프롬프트에 시간 스탬프를 붙여 특정 시점에 이벤트를 일어나게 하는 실험적 기능도 소개되었으나, 일반 사용자 인터페이스에서는 지원되지 않을 수 있습니다. 따라서 중요한 사건 전환은 차라리 별도 프롬프트로 나눠 생성한 후 편집으로 연결하는 편이 현실적입니다.
- **자연어 문장 vs 키워드 나열**: 영상 프롬프트에서는 완전한 문장 형태의 묘사가 권장되는 경향이 있습니다. Runway 가이드에 따르면, 키워드만 던지는 것보다 문장으로 맥락을 주면 모델이 각 요소 간 관계를 더 잘 파악한다고 합니다. 예를 들어 "A busy street during rain, people rushing with umbrellas as cars splash water onto the sidewalk."처럼 쓰면, 키워드 나열보다 장면 이해에 도움이 됩니다. 한편 Pika Labs 등 일부 서비스는 특정 키워드 (예: "cinematic", "4K")에 반응하도록 튜닝되었으므로, 해당 플랫폼 커뮤니티의 팁을 참고해 키워드와 문장을 조합하면 좋습니다. 전반적으로는 문법적으로 완결된 묘사 속에 중요한 키워드를 포함시키는 방식을 추천합니다.
- **장면 전환과 다중 클립**: 현 단계에서 하나의 텍스트 프롬프트로 복수의 장면 전환이 있는 긴 영상을 생성하는 것은 제한적입니다. 대부분 서비스는 수 초 길이의 하나의 시퀀스를 만들기 때문입니다. 따라서 스토리가 있는 영상을 만들려면 장면별로 별도 프롬프트를 작성하여 여러 영상을 생성하고 이를 후편집해야 합니다. 이 경우 각 장면 프롬프트에 동일한 주요 인물/스타일을 언급하여 연속성을 주는 것이 좋습니다. 또한 몇몇 도구는 이전 클립의 마지막 프레임을 다음 프롬프트의 참조 이미지로 사용하게 하여 (Runway의 이미지-투-비디오 기능 등) 부드러운 연결을 도모할 수 있습니다. 예를 들어 첫 장면 마지막 이미지를 추출해 두 번째 장면 생성 시 "Use this image as starting frame" 기능을 쓰면 동일 인물/환경 유지에 효과적입니다.
- **모델/도구별 특징**: OpenAI Sora는 사실적인 영상과 애니메이션 모두 우수하지만 여전히 제한된 그룹에 공개되어 있고, Pika Labs는 디스코드 플러그인 형태로 스타일리시한 짧은 영상을 잘 만들며 프롬프트에 카메라 움직임 등을 적절히 반영합니다. Runway Gen-2는 웹 UI에서 누구나 사용 가능하며 자연어 입력을 잘 해석하고, 특히 카메라 용어와 필름 스타일 지시에 강점이 있습니다. 예를 들어

Runway에서는 "handheld documentary film style, natural camera shake" 같은 프롬프트가 실제 핸드헬드 느낌을 잘 살렸다는 평가를 받았습니다. 한편으로 Google의 Model (예: Phenaki) 등은 프롬프트에 시간축 표현을 실험 중인데, 일반 공개는 제한적입니다. 사용 중인 플랫폼의 문서와 커뮤니티 팁을 확인하여 가장 효과적인 프롬프트 패턴을 따르는 것이 중요합니다.

- **예시: 영상 프롬프트 구조 예 (Runway 권장 방식)** "[카메라 슷] of [주제] [동작] in [환경]. [추가 시각/동작 설명, 스타일]..." 예시: "Medium shot of a cowboy perched on a horse in a dusty desert. The horse rears violently, cowboy falling off to the left. Backlit, western cinematic, high contrast, golden hour dust." – 이 프롬프트에서는 카메라 구도(중간거리 슷), 주제(카우보이와 말), 배경(먼지 나는 사막), 동작(말이 서고 카우보이가 떨어짐), 스타일/분위기(역광, 서부극 영화풍, 황금빛 저녁) 등을 한데 담고 있습니다. 이렇게 작성하면 모델이 시각적 정보와 시간적 흐름을 모두 파악하여 원하는 장면의 영상을 생성합니다.

마지막으로, 영상 프롬프트도 반복적 개선 과정이 필요합니다. 첫 결과가 마음에 들지 않으면 프롬프트를 수정하거나, 모델 파라미터(예: 시드 값)를 바꿔 재시도해보세요. 텍스트-투-비디오 기술은 이미지에 비해 아직 제약이 많아 완벽한 제어는 어렵지만, 프롬프트를 통해 원하는 분위기와 핵심 동작을 최대한 구체적으로 전달하면 보다 기대에 가까운 결과를 얻을 수 있습니다

## 6. PPT 생성을 위한 프롬프팅 전략: AI 프레젠테이션 생성

대화형 LLM과 오피스 제품의 결합으로, 자연어 프롬프트만으로 파워포인트(PPT)나 슬라이드 deck을 생성하는 시대가 되었습니다. Microsoft 365 Copilot의 PowerPoint, Google 슬라이드의 Duet AI(Gemini), SlidesGPT 등 다양한 도구가 텍스트 지시에 따라 프레젠테이션 초안을 자동 생성합니다. 이러한 AI 슬라이드 생성에서 유용한 프롬프트 작성 전략을 알아보겠습니다.

- **자료 출처 및 입력 제공:** 슬라이드 생성을 요청할 때, 기반이 될 내용을 함께 제공하는 것이 중요합니다. 예를 들어 이미 작성된 기획서 문서나 보고서가 있다면 해당 텍스트를 읽어 슬라이드를 만들도록 할 수 있습니다. Microsoft Copilot의 경우 "이 Word 문서를 기반으로 발표 자료 만들어줘"라고 할 수 있고, Google Slides의 Duet AI도 "다음 아웃라인을 슬라이드로 만들어줘: ..."를 지원합니다. 프롬프트에 OneDrive/드라이브 파일 링크나 본문 텍스트를 포함하여 "이 내용만 사용해서 만들어줘, 추가 추론은 하지 마"라고 명시하면 사실과 다른 내용 생성(환각)을 막는데 도움이 됩니다.
- **슬라이드 구조와 분량 지정:** 몇 장의 슬라이드를 원하는지, 대략 어떤 구성(agenda)으로 나누고 싶은지를 프롬프트에 담으세요. 예를 들어 "10장 분량의 투자 제안서를 만들어줘. 1장은 소개, 2~3장은 문제 정의, 4~6장은 솔루션, 7장은 로드맵, 8장은 팀 구성, 9장은 재무 계획, 10장은 결론으로 해줘."처럼 가이드라인을 줄 수 있습니다. 혹은 간단히 "약 5장 정도의 슬라이드"라고 해도 됩니다. 슬라이드 수를 지정하면 AI가 내용을 분량에 맞게 분배하며, 구성을 제시하면 그에 따라 챕터를 나눠 작성합니다.

- **청중과 목적 명시:** 슬라이드 내용의 대상 청중(audience)과 목적(outcome)을 알려주면 톤과 내용 선정이 정확해집니다. 예를 들어 "대상: 마케팅 비전문가 경영진, 목적: 캠페인 아이디어 승인" 식으로 써두면, 전문용어 사용을 줄이고 설득 강조점을 맞춰줍니다. B2B 고객 대상 발표라면 더 포멀하게, 내부 팀 공유용이면 캐주얼하게 등 톤을 결정할 수 있습니다. Google Gemini 가이드에서도 프롬프트에 페르소나(역할)와 맥락을 포함할 것을 권장하고 있으며, 이런 정보는 슬라이드 작성에도 유용합니다.
- **디자인 스타일과 브랜드 요소:** AI에게 슬라이드 디자인의 스타일이나 브랜드 템플릿을 지정할 수도 있습니다. Copilot에는 "우리 회사 템플릿 사용해줘" 같은 지시를 할 수 있고, Beautiful.ai 등의 도구도 "modern minimalist style" 같은 키워드를 인식합니다. 따라서 프롬프트에 "회사 브랜드 컬러인 파랑/회색 사용", "미니멀한 테마 적용", "시각화는 아이콘 위주로" 등 디자인 선호를 적으면 결과에 영향을 줍니다. 단, 아직 AI가 디자인 미적 판단은 한계가 있으니, 너무 추상적인 미감 표현은 피하고 구체적인 지침 (예: 배경 밝게, 글자 크게, 글머리 기호 사용 등)을 주는 것이 안전합니다.
- **콘텐츠 형태 지침:** 슬라이드에는 제목과 요점(Bullet points)이 핵심이므로, 프롬프트에서 한 슬라이드당 몇 개의 요점을 원한다든지, 문장보다는 키워드 위주로 적으라든지 요구할 수 있습니다. 예를 들어 "각 슬라이드에는 3~4개의 짧은 bullet point만 사용하고, 1줄 이상의 문장은 피해주세요." 같은 지시로 과도한 텍스트를 방지할 수 있습니다. 또한 "2장에는 목차(agenda) 슬라이드 포함, 마지막 장에는 Q&A 준비 슬라이드 추가"처럼 일반적인 발표 구조 요소를 넣도록 요구하세요. Microsoft Copilot 예시에서는 자동으로 목차와 요약 슬라이드를 추가해주지만, 그렇지 않은 도구도 있으니 안전하게 프롬프트에 언급하는 것이 좋습니다.
- **발표 노트와 부가 자료:** AI가 발표자 노트(발표 스크립트)까지 생성할 수 있다면 프롬프트로 요구할 수 있습니다. 예: "각 슬라이드마다 발표자 노트를 2~3문장씩 작성해줘." Copilot에서는 이러한 요청이 가능하며, SlidesGPT 등도 노트를 생성해줍니다. 다만 노트 생성은 불필요한 세부사항까지 만들어낼 수 있으니 "짧게 요점만." 등으로 통제하세요. 또한 차트나 이미지가 필요하면 "3장에는 매출 증가 추이를 나타내는 막대그래프 삽입" 또는 "4장에 AI 개념을 보여줄 아이콘형 일러스트 추가"처럼 프롬프트에 지시 가능합니다. Google Slides의 경우 "이미지 생성" 기능과 연계하여 "Slide 5: '신제품 전시 부스' 이미지를 생성하여 배경으로 삽입(텍스트 캡션: '첨단 제품 전시 현장')" 이런 식으로도 프롬프트를 구성할 수 있습니다.
- **추가 편집과 확인:** AI가 생성한 초기 슬라이드를 받으면, 후속 프롬프트로 미세 조정을 할 수 있습니다. 예를 들어 Copilot에게 "슬라이드 수를 15장으로 줄이고 한 슬라이드당 한 가지 핵심 메시지만 남겨줘."라고 하면 내용을 압축해줄 수 있습니다. 또는 "bullet들을 문장 대신 키워드로 바꿔줘.", "오탈자나 어색한 문장 고쳐줘."처럼 개선 지시를 내립니다. 대부분의 프레젠테이션 AI 도구는 대화형으로 추가 요청을 받으므로, 이를 활용해 다듬고 수정해서 완성도를 높입니다. 물론, 최종적으로 중요 수치나 사실은 사람이 검토해야 하며, 필요에 따라 슬라이드 순서를 재배치하거나 디자인을 손볼 수도 있습니다.

PPT 생성 프롬프트의 핵심은 "사용자가 원하는 최종 프레젠테이션의 청사진"을 텍스트로 잘 묘사하는 것입니다. 인간이라면 당연히 고려할 요소들(청중, 목적, 구성, 디자인)을

빠짐없이 포함하여 지시하면 **AI**도 그 틀 안에서 최대한 결과물을 맞춰줍니다. 아직 완벽하진 않아도, 이러한 구체적 프롬프트 덕분에 **AI**는 초안을 **70~80%** 수준까지 만들어주고, 사용자는 나머지 **20~30%**를 편집하며 시간을 크게 절약할 수 있습니다. 특히 반복적인 사내 보고서나 템플릿화된 제안서 작성에 이 방식을 응용하면 생산성이 향상될 것입니다.

## 6. 기타 목적을 위한 프롬프팅 전략

LLM과 생성 **AI**는 상상 이상의 다양한 용도로 활용될 수 있습니다. 이 장에서는 앞서 다룬 범주 이외에 자주 거론되는 몇가지 사용 사례 – 예컨대 엑셀 자동화, 시뮬레이션 생성, 교육 콘텐츠 개발, 개인화된 답변 생성 등 – 에 대해 프롬프트 작성 전략과 팁을 간략히 소개합니다. 각 용도별로 어떤 점에 중점을 두면 좋은 결과를 얻을 수 있는지 정리했습니다.

- **엑셀 자동화 및 분석 프롬프트**: LLM을 이용해 **Excel** 작업을 자동화하거나 데이터 분석을 지시할 때는, 대상 데이터 범위와 수행할 조치를 정확히 지정해야 합니다. 예를 들어 **Microsoft 365 Copilot**의 **Excel**에서는 **=COPILOT("분류해줘", A1:A100)** 같은 수식으로 해당 범위 데이터를 분류하도록 할 수 있는데, 이때 프롬프트 인자에 **"A열의 고객 피드백을 긍정/부정으로 분류해줘"**처럼 명확한 작업 지침을 넣어야 합니다. 또한 프롬프트에 결과 형식을 요구하면 (예: **"표 형태로 출력"**) 모델이 바로 표나 목록으로 답을 넣어줍니다. **Excel**용 **AI** 프롬프트의 팁을 요약하면: ① 분석 대상 범위와 시트명 세부 지칭, ② 계산 또는 분석 방법 명시 (합계, 평균, 회귀 등 용어), ③ 결과 표현 방식 지시 (값만, 셀 강조, 그래프 생성 등), ④ 필요하면 조건이나 기준 포함 (예: **"매출 > 100만인 행만 필터링"**). 예시: **"B열 (매출액)의 합계를 구하고, 그 값을 셀 F1에 굵게 표시해주세요."** 이렇게 쓰면 **AI**가 **B열** 합계 수식을 넣거나 값을 계산해줍니다. 현재 **Copilot Excel** 기능은 보안 등 이유로 웹검색은 못하고 주어진 시트 내에서만 일하므로, 외부 정보 요구는 피해야 합니다. 마지막으로 민감한 데이터는 **AI**가 학습에 사용하지 않음을 **Microsoft**가 명시했지만, 기업 정책에 따라 꺼릴 수 있으니 주의합니다.
- **시뮬레이션/시나리오 생성 프롬프트**: LLM을 활용해 가상의 시나리오를 시뮬레이션하거나 "만약 **X**라면 **Y**" 상황을 연습할 때는 상황과 역할을 구체적으로 설정하는 것이 중요합니다. 예를 들어 "당신은 리테일 매장의 매니저입니다. 이번 분기 매출이 **20%** 하락한 상황에서, 다음 분기 매출을 늘리기 위한 전략을 어떻게 세우겠습니까?"처럼 배경 설정(리테일 매니저, 매출 하락)과 과제(전략 수립)를 프롬프트에 담습니다. 이렇게 하면 모델이 해당 역할에 맞춰 현실적인 대응을 생성합니다. 시나리오 프롬프트 팁: ① 역할/인물 지정 ("너는 ~~이다"), ② 환경/상황 상세 묘사 ("~~한 상황에 놓여있다"), ③ 문제나 목표 제시 ("어떻게 ~할 것인가?"), ④ 필요하면 동적으로 변화하는 변수 추가 ("만약 ~이 발생하면?"). 또한 시뮬레이션을 상호작용 형태로 진행하려면 한 번에 길게 하기보다, 단계별 프롬프트로 이어가는 게 좋습니다. 예를 들어 먼저 초기 상황에 대한 대처를 묻고, 그 답변을 받은 뒤 "그럼 이런 돌발 상황이 추가로 생겼다. 어떻게 할 것인가?"를 이어서 물어보는 식입니다. 이를 통해 모델은 앞 맥락을 유지하며 동적으로 시나리오에 대응합니다. 이런 시나리오 기반 프롬프팅은 비즈니스 전략 시뮬, 고객 응대 트레이닝, 위기 대응 연습 등 다양한 분야에 활용 가능하며, **AI**의 맥락 인지 능력을 높여주는 효과가 있습니다.
- **교육 콘텐츠 개발 프롬프트**: LLM은 학습용 자료 제작에도 유용합니다. 이 때는 학습자의 수준과 학습목표를 프롬프트에 명시하는 것이 핵심입니다. 예를 들어 "중학교 2학년 수준으로, 작용-반작용 법칙을 설명하는 30분 분량의 과학 수업

계획을 만들어줘."처럼 대상 학년/연령, 주제, 형식(수업계획) 등을 모두 적습니다. 그러면 모델이 그 수준에 맞는 용어로 개념을 설명하고, 적절한 실험이나 활동 아이디어도 제안할 수 있습니다. 또 "이해도 체크를 위한 퀴즈 3문항 포함"처럼 부가 요구를 넣으면 퀴즈도 생성합니다. 교육 콘텐츠 프롬프트 팁: ① 콘텐츠 형태 지정 (강의노트, 대화형 연습문제, 시험지 등), ② 주제 범위 명시 ("일차방정식 기본 개념만 다룸"), ③ 어조/전개방식 ("재미있는 비유를 사용", "소크라테스식 문답으로"), ④ 분량 또는 난이도 ("초보자용 한글 5문장 설명", "고급자를 위한 심화 예제 2개 포함") 등을 포함하세요. 예시: "영어를 갓 시작한 한국인 초등학생을 위해, '간단한 자기소개'를 주제로 5분짜리 짧은 대화문과 한글 해석을 만들어줘. 새로운 단어 5개를 포함하고, 각 단어의 뜻도 주석으로 달아줘." 이렇게 하면 교육용으로 적합한 결과를 얻을 수 있습니다. 마지막으로 생성된 콘텐츠는 사실 오류나 부적절한 내용이 없는지 검토해야 합니다. 특히 역사나 과학 자료는 AI가 잘못된 정보를 섞을 수 있으니 전문가 검수가 필요합니다.

- **개인화된 답변 생성 프롬프트**: AI를 사용하여 개인 맞춤형 답변(예: 개인 이메일 초안, 고객별 응대문 등)을 만들 때는, 사용자/고객의 프로필이나 상황 정보를 프롬프트에 최대한 반영해야 합니다. 예를 들어 "상황: 고객 A는 배송 지연으로 불만. 역할: 당신은 고객지원 담당자. 고객 이름: 김민주, VIP 등급. 요청: 김민주 고객에게 배송 지연에 대해 정중히 사과하고, 보상으로 쿠폰 제공을 제안하는 3문장 이메일 초안을 작성해줘." 같이 씁니다. 여기서 고객 이름, 등급, 감정 상태, 원하는 톤(정중히 사과) 등을 모두 제공했으므로 모델은 이를 포함한 개인화된 이메일을 생성할 것입니다. 개인화 프롬프트 팁: ① 개인 정보 (이름, 직책, 선호 호칭 등) 명시, ② 이전에 주고 받은 내용 요약 (이전 대화 맥락을 반영하면 더 좋음), ③ 목소리 톤/스타일 ("친근한 말투로", "공식적 비즈니스 어조로"), ④ 포함 또는 제외할 내용 ("지난 주문 상세 언급", "사과문구 1회 이상 포함" 등) 등을 넣으세요. 또 다른 예로, 개인 취향에 맞춘 조언 글을 쓰게 할 때 "사용자 프로필: 35세 남성, IT 엔지니어, 초보 주자. 이 사람을 대상으로 5km 달리기 훈련 팁을 주세요."처럼 상황을 알려주면 일반적인 조언보다 개인화된 팁이 나옵니다. OpenAI Custom Instructions나 시스템 메시지를 활용하면 매번 프롬프트에 쓰지 않고도 사용자 프로필을 유지할 수 있지만, 일반 대화에서는 프롬프트에 직접 개인 맥락을 주입하는 것이 확실합니다. 끝으로, 개인화된 답변은 사생활 정보가 포함되므로 프라이버시 유의해야 합니다. AI가 보관하지 않도록 하는 한편, 민감정보는 프롬프트에 과도하게 넣지 않는 것이 좋습니다.

## 7. GPTs 지침

### [역할]

너는 사용자의 요청을 분석하여

LLM이 오해 없이 최적의 결과를 생성할 수 있도록

프롬프트를 설계·재작성하는 프롬프트 작성 챗봇이다.

### [지식 참조 원칙]

- 사용자의 프롬프트 작성 요청에 대한 모든 판단과 근거는

반드시 "지식"에 첨부된 파일만을 참조하여 수행한다.

- 첨부 파일에 근거하지 않은 추론, 일반 지식, 외부 상식은 사용하지 않는다.
- 첨부 파일에 근거가 없는 경우, 이를 명시하고 추론을 중단한다.

### **【핵심 목표】**

사용자의 한국어 입력이 애매할 경우,  
질문을 그대로 사용하지 말고  
사용자의 의도를 추론하여  
의미가 고정된 명확한 프롬프트로 변환한다.

### **【기본 원칙】**

1. 애매한 한국어 표현(괜찮아?, 될까?, 쓸 수 있어?, 애매한데?)은 그대로 프롬프트에 사용하지 않는다.
2. 번역을 하지 말고, 질문의 '의도'를 명시적 과업으로 재설계한다.
3. 질문에는 반드시 '무엇을 판단/생성/비교/설명할지'가 드러나야 한다.
4. 프롬프트 작성 근거는 반드시 첨부된 지식 파일에 근거하여 설명한다.
5. 프롬프트의 세부 구조와 형식은 지식 파일에 정의된 내용을 따른다.

### **【애매한 입력 판단 기준】**

다음 표현이 포함되면 '의미 불명확 상태'로 간주한다.

- 괜찮아?, 될까?, 쓸 수 있어?
- 애매한데?, 문제 없어?
- 이거 어때?, 좋은지 모르겠어

### **【의도 추론 규칙】**

의미가 애매한 경우, 아래 목적 중 가장 가능성 높은 것을 선택한다.

(복수 가능 시 실무·의사결정 목적을 우선한다)

1. 평가 (사용 가능 여부, 문제점 판단)
2. 비교 (A vs B)



3. 개선 (어떻게 고칠지)
4. 설명 (개념 이해)
5. 생성 (이미지, 영상, PPT, 인포그래픽 등)

#### **[프롬프트 유형 분기 규칙]**

사용자의 목적에 따라 프롬프트 생성 방식을 명확히 구분한다.

##### **1) LLM 답변을 원하는 경우**

- 프롬프트는 한국어로만 생성한다.
- 분석, 요약, 설명, 평가, 비교, 전략 수립 등이 이에 해당한다.
- 출력 언어도 한국어로 고정한다.

##### **2) 이미지 / 영상 / PPT / 인포그래픽 등 생성물을 원하는 경우**

- 프롬프트를 한국어와 영어 두 가지로 생성한다.
- 두 프롬프트는 의미와 의도가 동일해야 한다.
- 영어 프롬프트는 생성 품질 향상을 위한 용도로 사용한다.

#### **[프롬프트 재작성 원칙]**

1. 추론된 의도를 명확하고 단일한 과업으로 고정한다.
2. 평가·판단 요청일 경우, 기준이 누락되어 있으면 자동으로 보완한다.
3. 감정, 완곡, 추상 표현은 제거하고 기준 기반 요청으로 변환한다.
4. 질문이 애매할수록 프롬프트는 더 명확하고 구체적이어야 한다.

#### **[근거 설명 규칙]**

- 프롬프트 결과물 하단에  
“프롬프트 작성 근거”를 반드시 포함한다.
- 근거는 첨부된 지식 파일의 내용에 기반하여 설명한다.
- 개인 의견, 일반 상식, 추론처럼 보이는 표현은 사용하지 않는다.

#### **[시각화 자료 요청 처리 규칙]**

사용자가 이미지, 영상, PPT, 인포그래픽 등

시각화 자료 생성을 요청한 경우:

1. 생성 목적에 맞는 프롬프트를 먼저 작성한다.

2. 웹 검색을 통해

생성될 결과물과 유사한 실제 시각적 개체를 찾아

참고용으로 함께 제시한다.

3. 시각화 자료는 이해를 돕기 위한 참고용이며,

프롬프트 자체를 대체하지 않는다.

#### **[추가 질문 규칙]**

아래 조건이 모두 충족될 때만 사용자에게 추가 질문을 한다.

- 도메인 식별이 불가능한 경우

- 의도 추론이 불가능한 경우

- 결과 사용 목적에 따라 프롬프트가 크게 달라질 수 있는 경우

추가 질문은 반드시 선택형으로 제시한다.

#### **[절대 금지]**

- 첨부 지식 파일 외 정보를 근거로 사용하는 것

- 애매한 질문을 그대로 프롬프트에 포함하는 것

- “괜찮은지”, “될지” 같은 표현을 유지하는 것

- LLM 답변용 프롬프트에 영어를 포함하는 것

- 생성물용 프롬프트를 한국어만으로 제공하는 것

- 지식 파일에 정의된 프롬프트 구조를 임의로 재정의하는 것

#### **[최종 행동 원칙]**

사용자의 질문이 애매할수록,

질문을 그대로 쓰지 말고

지식 파일에 근거하여

의도와 판단 기준이 명확히 고정된 프롬프트로 재설계하라.